

Sổ tay chuẩn hóa project website giao bài từ xa

Backend .NET - Frontend Next.js - Supabase PostgreSQL - Cloudflare

Lưu ý quan trọng

Tài liệu này thiết kế hệ thống học liệu theo hướng hợp pháp: chỉ sử dụng tài liệu có giấy phép, tài liệu tự tạo hoặc nguồn được phép. Không hướng dẫn crawl/sao chép sách có bản quyền trái phép.

00 - Project Brief

Tên dự án

Remote Assignment Platform - Web giao bài, nộp bài và chấm điểm từ xa.

Mục tiêu

Xây dựng một hệ thống giúp giáo viên/manager tạo bài tập, giao bài theo lớp/khối, nhận bài nộp từ học sinh, chấm điểm theo thang 100, gửi thông báo cho phụ huynh/giáo viên, hỗ trợ học sinh bằng tài liệu, video, chat realtime và công cụ AI.

Người dùng chính

- Admin: quản trị hệ thống, cấu hình vai trò, dữ liệu nền, tài khoản.
- Manager: quản lý lớp, giáo viên, học sinh, bài tập, thống kê.
- Student: xem bài, làm bài, nộp bài, xem điểm/comment, chat hỏi bài.
- Parent: nhận thông báo, xem tiến độ, điểm, cảnh báo trễ hạn.

Tính năng trọng tâm

- Tạo bài tập từ nhập tay, Markdown, text, docx.
- Công cụ vẽ hình, chụp ảnh, viết text khi làm bài.
- Nộp bài bằng hình ảnh, text, docx.
- Chấm điểm thang 100, nhận xét rõ ràng.
- Email SMTP báo có bài nộp/cảnh báo.
- Nhắc nhở tự động khi sắp đến hạn hoặc trễ hạn.
- Realtime chat giữa học sinh và giảng viên/manager.
- Dashboard điểm, thời gian làm bài, lịch sử thao tác.
- Video bài giảng/hướng dẫn.
- Kho học liệu hợp pháp, chia theo lớp 6-12 và sách nâng cao nếu có quyền sử dụng.

Tiêu chí thành công

- Người dùng thao tác được rõ ràng, exception/error message dễ hiểu.
- Code dễ bảo trì, phân tầng tốt, không hard-code nghiệp vụ.
- Token, role, session, soft delete được xử lý an toàn.
- Deploy có tài liệu rõ cho Cloudflare + backend .NET.
- Có đủ docs để AI agent/code assistant làm đúng định hướng.

01 - Requirements

Functional requirements

Account & role

- Hệ thống có 4 vai trò: Admin, Manager, Student, Parent.
- Tài khoản chỉ được khóa mềm khi xóa/sửa trạng thái, không hard delete.
- Hỗ trợ mapping Parent -> Student để phụ huynh xem tiến độ của con.
- JWT access token mặc định 15 phút cho Admin/Manager/Parent.
- Student access token tối đa 1 giờ, nhưng vẫn cần session tracking và refresh token rotation.

Assignment authoring

- Tạo bài tập bằng nhập tay.
- Import nội dung từ `.md`, `.txt`, `.docx`.
- Hỗ trợ thêm hình ảnh, file đính kèm, video hướng dẫn, tài liệu tham khảo Anh/Việt.
- Có công cụ vẽ hình/canvas cho bài cần hình học, sơ đồ, ghi chú trực quan.
- Có cấu hình giao bài tự động theo lịch.

Submission

- Student nộp bài bằng text, ảnh, docx hoặc file đính kèm.
- Ghi log thời gian bắt đầu, thời gian chỉnh sửa, thời gian nộp.
- Cho phép comment bài nộp và phản hồi từ giảng viên.
- Trạng thái: Draft, Submitted, Late, Graded, Returned, Resubmitted.

Grading

- Điểm theo thang 100.
- Cho phép rubric theo tiêu chí.
- Giáo viên có thể ghi lời giải mẫu, nhận xét, file đính kèm.
- AI chỉ hỗ trợ gợi ý, quyết định cuối cùng thuộc về giáo viên/manager.

Notification & email

- SMTP gửi email cho giáo viên/manager khi có bài nộp.
- Gửi email cho phụ huynh khi học sinh trễ hạn, điểm thấp, hoặc có nhận xét quan trọng.
- Nhắc học sinh trước hạn, khi quá hạn và khi có bài mới.

Realtime communication

- Chat realtime giữa học sinh và giáo viên/manager theo lớp/bài tập.
- Hỗ trợ unread count, message status, moderation log.

Learning resources

- Tài liệu được chia theo khối/lớp/môn/chương/bài.
- Hỗ trợ tài liệu tiếng Việt và tiếng Anh.
- Sách/tài liệu chỉ được đưa vào hệ thống khi có quyền sử dụng hợp pháp.
- Viewer tải từng trang hoặc từng chunk để giảm tải, không render toàn bộ file một lần.

Non-functional requirements

- Bảo mật: JWT validation đầy đủ, refresh token rotation, rate limit, audit log.
- UX: lỗi phải rõ, có code lỗi, thông điệp người dùng và hướng xử lý.
- Performance: phân trang, lazy loading, upload file theo giới hạn kích thước.
- Maintainability: backend theo Clean Architecture nhẹ, frontend theo feature module.
- Observability: structured logs, audit logs, dashboard lỗi.
- Compliance: không crawl/phân phối tài liệu có bản quyền trái phép.

02 - Scope and Legal Boundaries

Phạm vi được làm

Hệ thống được phép hỗ trợ:

- Giáo viên tự upload tài liệu do mình tạo.
- Tài liệu open educational resources có giấy phép rõ ràng.
- Tài liệu nhà trường có quyền sử dụng.
- Tài liệu mua bản quyền và được phép số hóa theo hợp đồng.
- Link tham khảo tới nguồn chính thống nếu nguồn không cho phép copy.

Không làm

Không thiết kế hoặc triển khai:

- Cào/copy toàn bộ sách giáo khoa có bản quyền khi chưa có quyền.
- Vượt cơ chế chống tải, chống sao chép, DRM hoặc giới hạn truy cập.
- Tự động tải tài liệu từ nguồn không cấp phép.
- Phát tán PDF sách bản quyền cho người không được cấp quyền.

Thiết kế thay thế hợp pháp

Thay vì “cào toàn bộ sách”, hệ thống nên có Legal Content Ingestion Pipeline:

1. Admin/Manager tạo nguồn tài liệu.
2. Nhập metadata: nhà xuất bản, giấy phép, phạm vi dùng, lớp, môn, năm.
3. Upload file hoặc nhập link chính thống.
4. Tách tài liệu thành page/chunk nếu giấy phép cho phép.
5. Tạo index tìm kiếm nội bộ.
6. Log người xem/tải.
7. Cho phép gỡ tài liệu khi hết quyền sử dụng.

Viewer từng trang

Viewer có thể load từng trang để:

- Giảm RAM/tránh tải file lớn một lần.
- Tăng tốc độ mở tài liệu.
- Dễ phân quyền theo trang/chương.
- Ghi log học tập theo page view.

Không mô tả viewer như công cụ “né IDM” hoặc “chống download tuyệt đối”. Trình duyệt luôn có thể tải nội dung đã được gửi đến client; cách đúng là phân quyền, watermark, audit log, rate limit và chỉ phát hành tài liệu hợp pháp.

03 - Role & Permission Matrix

Role summary

```
| Role | Mục đích |
|-----|-----|
| Admin | Quản trị toàn hệ thống, cấu hình, khóa/mở tài khoản, nhân quyền |
| Manager | Quản lý lớn, học sinh, bài tập, điểm, báo cáo |
| Student | Nhân bài, làm bài, nộp bài, xem điểm/comment, chat hỏi bài |
| Parent | Xem tiến độ, điểm, cảnh báo, thông báo của con |
```

Permission matrix

```
| Module | Admin | Manager | Student | Parent |
|-----|-----|-----|-----|-----|
| Quản lý tài khoản | Full | Limited | No | No |
| Khóa/mở tài khoản | Yes | Limited | No | No |
| Quản lý lớn/khối | Full | Full | Read own | Read child |
| Tạo bài tập | Yes | Yes | No | No |
| Làm bài/nộp bài | No | No | Yes | No |
| Chấm bài | No/Config | Yes | No | No |
| Xem điểm | Full | Full assigned | Own | Child only |
| Comment bài làm | Yes | Yes | Own thread | Child view/comment tùy cấu hình |
| Chat realtime | Moderation | Assigned rooms | Own rooms | Parent channel |
| Quản lý sách/tài liệu | Full | Upload/Manage assigned | Read allowed | Read child allowed |
| Dashboard/log | Full | Assigned scope | Own | Child only |
| SMTP config | Full | No | No | No |
```

Rule phân quyền

- Không check role trực tiếp rải rác trong controller nếu nghiệp vụ phức tạp.
- Dùng policy-based authorization.
- Mọi query dữ liệu phải filter theo scope: tenant/class/student/parent relation.
- Parent không bao giờ được xem dữ liệu học sinh khác.
- Student không được truy cập bài chưa được giao hoặc tài liệu chưa được cấp quyền.

04 - Domain Model

Aggregate chính

User Account

- User
- Role
- UserSession
- RefreshToken
- ParentStudentLink
- AccountAuditLog

Education Structure

- GradeLevel
- Subject
- ClassRoom
- ClassEnrollment
- CourseUnit
- Lesson

Assignment

- Assignment
- AssignmentTarget
- AssignmentAttachment
- AssignmentSchedule
- AssignmentReminderRule
- AssignmentRubric
- AssignmentReference

Submission

- Submission
- SubmissionFile
- SubmissionCanvasSnapshot
- SubmissionComment
- SubmissionActivityLog

Grading

- Grade
- GradeRubricItem

- TeacherSolution
- FeedbackComment

Learning Resource

- ResourceCollection
- LearningResource
- ResourcePage
- ResourceAccessLog
- ResourceLicense

Realtime & notification

- ChatRoom
- ChatMessage
- Notification
- EmailOutbox
- NotificationPreference

Trạng thái bài tập

| State | Ý nghĩa |
 |-----|
 | Draft | Manager đang soạn |
 | Scheduled | Đã lên lịch giao |
 | Published | Đã giao |
 | Closed | Hết hạn nhận bài |
 | Archived | Lưu trữ |

Trạng thái bài nộp

| State | Ý nghĩa |
 |-----|
 | Draft | Student đang làm |
 | Submitted | Đã nộp |
 | Late | Nộp trễ |
 | Graded | Đã chấm |
 | Returned | Bị trả về yêu cầu sửa |
 | Resubmitted | Nộp lại |

Invariant nghiệp vụ

- Điểm cuối cùng nằm trong [0, 100].
- Một student chỉ có một active submission cho một assignment, trừ khi giáo viên cho phép resubmit.
- Assignment đã Published không được xóa cứng.
- Account bị khóa mềm không được login nhưng dữ liệu lịch sử vẫn giữ.
- Email gửi đi phải qua outbox để retry, không gửi trực tiếp trong transaction chính.

05 - Database Design

Database

Dùng Supabase PostgreSQL làm database chính.

Nguyên tắc thiết kế

- Tất cả bảng quan trọng có `id uuid`, `created\_at`, `updated\_at`.
- Soft delete dùng `deleted\_at`, `deleted\_by`, `is\_active` nếu cần.
- Dữ liệu nhạy cảm không lưu plain text.
- File không lưu trực tiếp trong DB; chỉ lưu metadata + storage key.
- Các bảng log/audit chỉ append, không update tùy tiện.
- Query phải có index theo `tenant\_id`, `class\_id`, `student\_id`, `assignment\_id`, `created\_at`.

Nhóm bảng chính

```
| Nhóm | Bảng |
|-----|
| Identity | users, roles, user_roles, user_sessions, refresh_tokens |
| School | grade_levels, subjects, class_rooms, class_enrollments |
| Assignment | assignments, assignment_targets, assignment_files, assignment_reminder_rules |
| Submission | submissions, submission_files, submission_comments, submission_activity_logs |
| Grading | grades, grade_rubric_items, teacher_solutions |
| Learning Resource | resource_collections, learning_resources, resource_pages, resource_access_logs |
| Communication | chat_rooms, chat_messages, notifications, email_outbox |
```

Supabase Row Level Security

Nếu backend .NET là API duy nhất truy cập database bằng service role/connection string server-side, authorization chính nằm ở backend. Nếu frontend truy cập Supabase trực tiếp, bắt buộc bật RLS và viết policies đầy đủ. Với dự án này, khuyến nghị frontend gọi backend .NET để kiểm soát nghiệp vụ nhất quán.

File storage

Có 2 hướng:

1. Supabase Storage: hợp khi muốn quản lý file gần Postgres và dùng RLS/policies.
2. Cloudflare R2: hợp khi file lớn, nhiều ảnh/docx/video, cần CDN tốt và chi phí egress thấp.

Khuyến nghị MVP: Supabase Storage cho đơn giản. Khi lớn: tách video/sách sang R2.

08 - System Architecture

Kiến trúc tổng thể

```
Browser
|
| HTTPS
v
Cloudflare DNS/WAF/CDN
|
+--> Frontend: Next.js on Cloudflare Pages/Workers
|
+--> Backend API: ASP.NET Core hosted on .NET-compatible compute
|
+--> Supabase PostgreSQL
+--> Supabase Storage / Cloudflare R2
+--> SMTP Provider
+--> AI Provider Gateway
+--> Background Worker
```

Lưu ý deploy .NET với Cloudflare

Cloudflare rất phù hợp làm DNS, CDN, WAF, Pages/Workers cho frontend và edge logic. Nhưng ASP.NET Core backend cần chạy trên môi trường hỗ trợ .NET runtime/container như VPS, Azure App Service, Fly.io, Render, Railway, Docker host hoặc server riêng. Sau đó đặt sau Cloudflare bằng DNS proxy, Cloudflare Tunnel hoặc Zero Trust.

Backend architecture

```
src/
├── WebApi
├── Application
├── Domain
├── Infrastructure
└── Worker
```

- Domain: entity, value object, domain rule.
- Application: use case, DTO, validation, interface.
- Infrastructure: EF Core/Dapper, Supabase storage, SMTP, AI adapter.
- WebApi: controller, auth, exception middleware.
- Worker: reminder, email outbox, scheduled assignment.

Frontend architecture

```
src/
├── app/
├── features/
├── components/
├── lib/
├── hooks/
├── styles/
└── types/
```

- `features/assignments`: giao bài.
- `features/submissions`: làm/nộp bài.
- `features/grading`: chấm điểm.

- `features/resources` : kho học liệu.
- `features/chat` : realtime chat.
- `features/dashboard` : biểu đồ/log.

Data flow: nộp bài

Student opens assignment
→ Backend creates/loads draft submission
→ Student edits text/canvas/uploads image/docx
→ Files go to storage
→ Metadata saved in DB
→ Student submits
→ Submission status = Submitted/Late
→ EmailOutbox creates teacher notification
→ Notification created for Manager/Teacher
→ Dashboard updates

Data flow: reminder

Worker scans due assignments
→ Finds students without submitted/graded submission
→ Creates notification
→ Enqueues email if configured
→ Logs reminder event

11 - Assignment Authoring

Input sources

Hệ thống hỗ trợ tạo bài từ:

- Nhập tay trên editor.
- Markdown `.md`.
- Text `.txt`.
- Word `.docx`.
- File đính kèm.
- Template có sẵn.

Assignment structure

```
Assignment
├── Title
├── Description
├── Content blocks
│   ├── Markdown block
│   ├── Text block
│   ├── Image block
│   ├── File attachment block
│   ├── Video block
│   └── Drawing prompt block
├── References
│   ├── Vietnamese resources
│   └── English resources
├── Rubric
├── Due date
├── Reminder rules
└── Target classes/students
```

Content block design

Không lưu toàn bộ nội dung như một string khổng lồ nếu cần tương tác nhiều. Nên tách block:

```
| Block type | Dữ liệu |
|-----|
| markdown | markdown content |
| text | plain text |
| image | storage file id |
| docx | storage file id ± extracted text |
| drawing | canvas prompt ± expected output |
| video | video url/file id |
```

Import docx

Flow:

1. Upload docx.
2. Backend validate file type/size.
3. Extract text + heading nếu cần.
4. Lưu file gốc vào storage.

5. Tạo content blocks từ text đã extract.
6. Cho manager preview và chỉnh sửa trước khi publish.

Drawing tool

Nên có 2 chế độ:

- Teacher drawing prompt: giáo viên vẽ mẫu/sơ đồ trong đề.
- Student answer canvas: học sinh vẽ hình/lời giải.

Lưu:

- JSON scene/canvas data để edit lại.
- PNG snapshot để giáo viên xem nhanh.
- Audit log autosave.

Auto assignment

Giao bài tự động cần:

- ``publish_at``
- ``due_at``
- target class/student
- timezone
- reminder rules
- status transition rõ ràng

Worker scan mỗi vài phút hoặc dùng scheduler tùy hạ tầng.

12 - Submission & Grading

Submission types

- Text answer.
- Image upload.
- Docx upload.
- Canvas drawing.
- Mixed submission: text + image + docx + drawing.

Submission flow

```
Open assignment
→ Create draft if not exists
→ Autosave text/canvas/file metadata
→ Validate required answer
→ Submit
→ Lock submitted version
→ Notify teacher/manager
```

Versioning

Mỗi lần nộp chính thức nên tạo version:

- version number
- submitted_at
- submitted_by
- status
- file snapshot

Nếu giáo viên cho nộp lại, tạo version mới, không ghi đè bản cũ.

Grading scale

Điểm theo thang 100.

```
0 = không làm/không hợp lệ
1-49 = chưa đạt
50-64 = đạt tối thiểu
65-79 = khá
80-89 = tốt
90-100 = xuất sắc
```

Rubric mẫu

```
| Tiêu chí | Điểm tối đa |
|-----|
| Đúng kiến thức | 40 |
| Trình bày rõ ràng | 20 |
| Liên luận/cách giải | 25 |
| Đúng định dạng/nộp đủ | 15 |
```

Teacher solution

Lời giải giảng viên phải có:

- Nội dung lời giải.
- Giải thích từng bước.
- File đính kèm nếu có.
- Video giải thích nếu có.
- Comment riêng cho học sinh nếu cần.

AI-assisted grading

AI có thể hỗ trợ:

- Tóm tắt bài làm.
- Gợi ý lỗi sai.
- Gợi ý nhận xét.
- So sánh với rubric.

AI không được tự động chốt điểm cuối cùng nếu chưa có review của giáo viên, trừ khi hệ thống được cấu hình rõ cho bài trắc nghiệm/đáp án khách quan.

13 - Learning Resource Library

Mục tiêu

Tạo kho học liệu online cho học sinh và giảng viên xem/tải tài liệu được phép sử dụng.

Phân loại

```
Khối/Lớp
→ Môn
→ Bộ sách/Nguồn
→ Chương
→ Bài
→ Tài liệu
→ Trang/Chunk
```

Ví dụ:

```
Lớp 6
→ Toán
→ Kết nối tri thức
→ Chương 1
→ Bài 1
→ Tài liệu hướng dẫn / Bài tập bổ trợ / Video
```

Metadata bắt buộc

- title
- grade_level
- subject
- source_name
- publisher/author nếu có
- license_type
- license_note
- uploaded_by
- storage_key
- allow_download
- allow_page_view
- expires_at nếu quyền sử dụng có thời hạn

Page/chunk viewer

Flow

```
User opens resource
→ Backend checks permission/license
→ Get page metadata
→ Client requests page 1
→ Lazy load next page on scroll
→ Log page view
```


Rules

- Không render toàn bộ sách/tài liệu lớn một lần.
- Có pagination/lazy loading.
- Có watermark nếu tài liệu nội bộ.
- Có rate limit download/page requests.
- Có audit log ai xem/tải lúc nào.

Download policy

- Nếu tài liệu được phép tải: cung cấp download chính thức.
- Nếu không được phép tải: chỉ cho xem theo quyền, nhưng không hứa “chống tải tuyệt đối”.
- Không thiết kế cơ chế né/phá công cụ download. Bảo vệ đúng hướng bằng phân quyền, điều khoản sử dụng, watermark, log và nội dung hợp pháp.

14 - AI Integration

Mục tiêu AI

AI hỗ trợ giáo viên, học sinh và manager, nhưng không thay thế vai trò chấm/duyet của giáo viên.

Use cases

Teacher/Manager

- Tạo đề bài từ markdown/docx/text.
- Tạo rubric thang 100.
- Tóm tắt bài nộp.
- Gợi ý nhận xét.
- Gợi ý bài tập tương tự.
- Dịch/tạo tài liệu tham khảo Anh/Việt.

Student

- Gợi ý hướng giải, không đưa đáp án trực tiếp nếu bài đang làm yêu cầu tự luận.
- Giải thích khái niệm.
- Tóm tắt video/tài liệu được phép.
- Kiểm tra lỗi chính tả/trình bày.

Parent

- Tóm tắt tiến độ học của con.
- Gợi ý cách hỗ trợ con học bài.

AI provider strategy

Không hard-code một nhà cung cấp. Tạo interface:

```
public interface IAiAssistantClient
{
    Task<AiResult> GenerateAsync(AiRequest request, CancellationToken ct);
}
```

Provider có thể là:

- Free-tier AI API nếu còn phù hợp tại thời điểm triển khai.
- Local model/Ollama nếu có máy chủ.
- Provider trả phí sau này.
- Rule-based fallback nếu AI hết quota.

Safety rules

- Không gửi dữ liệu nhạy cảm không cần thiết lên AI provider.
- Mask tên/email/số điện thoại nếu không cần.
- Log prompt/response ở mức phù hợp nhưng tránh lộ thông tin cá nhân.
- AI response phải được đánh dấu là gợi ý.
- Với chấm điểm tự luận, giáo viên phải review trước khi chốt.

Prompt template: hỗ trợ chấm bài

Bạn là trợ lý chấm bài. Hãy đọc đề, rubric và bài nộp.

Không tự chốt điểm cuối cùng. Chỉ gợi ý:

1. Ý đúng
2. Ý sai/thiếu
3. Gợi ý điểm theo từng tiêu chí
4. Nhận xét ngắn, dễ hiểu cho học sinh

18 - Cloudflare Deployment

Mục tiêu deploy

- Frontend Next.js deploy lên Cloudflare Pages/Workers.
- Backend .NET chạy trên host hỗ trợ .NET/container và đi qua Cloudflare.
- Supabase PostgreSQL là database cloud.
- SMTP provider cấu hình qua environment variables.

Kiến trúc deploy khuyên dùng

Cloudflare DNS

- app.example.com → Next.js frontend on Cloudflare
- api.example.com → ASP.NET Core backend host, proxied by Cloudflare
- assets.example.com → optional R2/Supabase public asset domain

Frontend deployment

- Dùng Cloudflare Pages/Workers theo hướng dẫn Next.js chính thức.
- Build command thường là `npm run build` hoặc command theo adapter đang dùng.
- Environment variables:

```
NEXT_PUBLIC_API_BASE_URL=https://api.example.com
NEXT_PUBLIC_APP_NAME=Remote Assignment Platform
```

Backend deployment

Vì backend là ASP.NET Core, cần host hỗ trợ .NET:

- Docker VPS.
- Azure App Service.
- Render/Railway/Fly.io nếu phù hợp.
- Server trường học/cá nhân + Cloudflare Tunnel.

Environment variables:

```
ASPNETCORE_ENVIRONMENT=Production
DATABASE_URL=postgresql://...
JWT_ISSUER=https://api.example.com
JWT_AUDIENCE=remote-assignment-api
JWT_SIGNING_KEY=change_me
SMTP_HOST=...
SMTP_USERNAME=...
SMTP_PASSWORD=...
SUPABASE_URL=...
SUPABASE_SERVICE_ROLE_KEY=...
```

Cloudflare Tunnel option

Nếu backend chạy trên máy/VPS riêng:

```
cloudflared tunnel  
→ api.example.com  
→ localhost:5000
```

Dùng khi muốn không mở port trực tiếp hoặc muốn đưa backend nội bộ ra internet qua Cloudflare.

CI/CD recommended

- Frontend: Cloudflare Git integration.
- Backend: GitHub Actions build Docker image/deploy host.
- Database: migration chạy bằng pipeline riêng, không chạy tự động bù đắp trên production.

Production checklist

- HTTPS only.
- CORS chỉ cho domain frontend.
- Rate limit login/upload/AI.
- Backup database.
- SMTP credentials trong secret manager.
- JWT key đủ mạnh và rotate được.
- Logging không lộ token/password.

Project Rules

Ưu tiên của project

1. Đúng nghiệp vụ giáo dục.
2. Bảo mật tài khoản, bài nộp, điểm số.
3. UX dễ hiểu cho học sinh/phụ huynh.
4. Code dễ bảo trì.
5. Có log/audit cho thao tác quan trọng.
6. Tài liệu/học liệu hợp pháp.

General rules

- Không hard delete tài khoản, bài tập, bài nộp, điểm.
- Không gửi email trực tiếp trong transaction chính.
- Không để AI tự chốt điểm nếu chưa cấu hình rõ.
- Không import/crawl tài liệu không có quyền sử dụng.
- Không để frontend quyết định quyền truy cập dữ liệu.
- Không để error message lộ stack trace/token/connection string.

Definition of Done

Một feature chỉ được xem là xong khi:

- Có validation.
- Có authorization.
- Có error message rõ.
- Có log/audit nếu là thao tác quan trọng.
- Có test happy path và error path.
- Có cập nhật docs nếu thay đổi nghiệp vụ.

AI Agent Rules

Role

Bạn là senior full-stack developer hỗ trợ xây dựng hệ thống web giao bài từ xa bằng .NET, Next.js, Supabase PostgreSQL và Cloudflare.

Must read first

Trước khi code, phải đọc:

1. `docs/00-project-brief.md`
2. `docs/01-requirements.md`
3. `docs/02-scope-and-legal-boundaries.md`
4. `docs/08-system-architecture.md`
5. Toàn bộ file trong `rules/`

Behavior rules

- Không tự ý crawl/copy sách có bản quyền.
- Không hard delete dữ liệu quan trọng.
- Không tạo endpoint thiếu authorization.
- Không lưu token/password trong log.
- Không dùng AI để tự động chốt điểm tự luận nếu chưa được yêu cầu rõ.
- Nếu tạo file mới, đặt đúng folder.
- Nếu sửa business rule, cập nhật docs liên quan.

Output after coding

Sau mỗi task, phải báo cáo:

1. File đã tạo/sửa.
2. Logic đã thay đổi.
3. Cách test.
4. Rủi ro/việc cần kiểm tra thủ công.
5. Docs cần cập nhật.

Preferred implementation order

1. Auth/session/role.
2. Database schema/migration.
3. Assignment CRUD.
4. Submission/upload.

5. Grading/comment.
6. Email outbox.
7. Reminder worker.
8. Realtime chat.
9. Dashboard.
10. AI helper.